A Technical Introduction to Weight initialization

Section 1

Others Instead of initializing all weights with random values on the order of O (1 / n) {\displaystyle O(1/{\sqrt {n}})} , sparse initialization initialized only a small subset of the weights with larger random values, and the other weights zero, so that the total variance is still on the order of O (1) {\displaystyle O(1)} . Random walk initialization was designed for MLP so that during backpropagation, the L2 norm of gradient at each layer performs an unbiased random walk as one moves from the last layer to the first. Looks linear initialization was designed to allow the neural network to behave like a deep linear network at initialization, since W R e L U (x) – W R e L U (– x) = W x {\displaystyle W\;\mathrm {ReLU} (x)-W\;\mathrm {ReLU} (-x)=Wx} . It initializes a matrix W {\displaystyle W} of shape R n 2 × m {\displaystyle \mathbb {R} ^{{\frac {n}{2}}\times m}} by any method, such as orthogonal initialization, then let the R n × m {\displaystyle \mathbb {R} ^{n\times m}} weight matrix to be the concatenation of W , – W {\displaystyle W,-W} .

Section 2

Initialize the classification layer and the last layer of each residual branch to 0. Initialize every other layer using a standard method (such as He initialization), and scale only the weight layers inside residual branches by L – 1 2 m – 2 {\displaystyle L^{-{\frac {1}{2m-2}}}} . Add a scalar multiplier (initialized at 1) in every branch and a scalar bias (initialized at 0) before each convolution, linear, and element-wise activation layer. Similarly, T-Fixup initialization is designed for Transformers without layer normalization.

Section 3

History Random weight initialization was used since Frank Rosenblatt's perceptrons. An early work that described weight initialization specifically was (LeCun et al., 1998). Before the 2010s era of deep learning, it was common to initialize models by "generative pre-training" using an unsupervised learning algorithm that is not backpropagation, as it was difficult to directly train deep neural networks by backpropagation. For example, a deep belief network was trained by using contrastive divergence layer by layer, starting from the bottom. (Martens, 2010) proposed Hessian-free Optimization, a quasi-Newton method to directly train deep networks. The work generated considerable excitement that initializing networks without pre-training phase was possible. However, a 2013 paper demonstrated that with well-chosen hyperparameters, momentum gradient descent with weight initialization was sufficient for training neural networks, without needing either quasi-Newton method or generative pre-training, a combination that is still in use as of 2024. Since then, the impact of initialization on tuning the variance has become less important, with methods developed to automatically tune variance, like batch normalization tuning the variance of the forward pass, and momentum-based optimizers tuning the variance of the backward pass. There is a tension between using careful weight initialization to decrease the need for normalization, and using normalization to decrease the need for careful weight initialization, with each approach having its tradeoffs. For example, batch normalization causes training examples in the minibatch to become dependent, an undesirable trait, while weight initialization is architecture-dependent.

Section 4

Miscellaneous For hyperbolic tangent activation function, a particular scaling is sometimes used: 1.7159 tanh ■ (2 x / 3) {\displaystyle 1.7159\tanh(2x/3)} . This was sometimes called "LeCun's tanh". It was designed so that it maps the interval [– 1 , + 1] {\displaystyle [-1,+1]} to itself, thus ensuring that the overall gain is around 1 in "normal operating conditions", and that | f ″ (x) | {\displaystyle |f''(x)|} is at maximum when x = – 1 , + 1 {\displaystyle x=-1,+1} , which improves convergence at the end of training. In self-normalizing neural networks, the SELU activation function S E L U (x) = λ { x if x > 0 α e x – α if x ≤ 0 {\displaystyle \mathrm {SELU} (x)=\lambda {\begin{cases}x&{\text{if }}x>0\\\alpha e^{x}-\alpha &{\text{if }}x\leq 0\end{cases}}} with parameters λ ≈ 1.0507 , α ≈ 1.6733 {\displaystyle \lambda \approx 1.0507,\alpha \approx 1.6733} makes it such that the mean and variance of the output of each layer has (0 , 1) {\displaystyle (0,1)} as an attracting fixed-point. This makes initialization less important, though they recommend initializing weights randomly with variance 1 / n l – 1 {\displaystyle 1/n_{l-1}} .

Section 5

Orthogonal initialization (Saxe et al. 2013) proposed orthogonal initialization: initializing weight matrices as uniformly random (according to the Haar measure) semi-orthogonal matrices, multiplied by a factor that depends on the activation function of the layer. It was designed so that if one initializes a deep linear network this way, then its training time until convergence is independent of depth. Sampling a uniformly random semi-orthogonal matrix can be done by initializing X {\displaystyle X} by IID sampling its entries from a standard normal distribution, then calculate (X X ■) – 1 / 2 X {\displaystyle \left(XX^{\top }\right)^{-1/2}X} or its transpose, depending on whether X {\displaystyle X} is tall or wide. For CNN kernels with odd widths and heights, orthogonal initialization is done this way: initialize the central point by a semi-orthogonal matrix, and fill the other entries with zero. As an illustration, a kernel K {\displaystyle K} of shape 3 × 3 × c × c ′

{\displaystyle 3\times 3\times c\times c'} is initialized by filling K [2 , 2 , : , :] {\displaystyle K[2,2,:,:]} with the entries of a random semi-orthogonal matrix of shape c × c ′ {\displaystyle c\times c'} , and the other entries with zero. (Balduzzi et al., 2017) used it with stride 1 and zero-padding. This is sometimes called the Orthogonal Delta initialization. Related to this approach, unitary initialization proposes to parameterize the weight matrices to be unitary matrices, with the result that at initialization they are random unitary matrices (and throughout training, they remain unitary). This is found to improve long-sequence modelling in LSTM. Orthogonal initialization has been generalized to layer-sequential unit-variance (LSUV) initialization. It is a data-dependent initialization method, and can be used in convolutional neural networks. It first initializes weights of each convolution or fully connected layer with orthonormal matrices. Then, proceeding from the first to the last layer, it runs a forward pass on a random minibatch, and divides the layer's weights by the standard deviation of its output, so that its output has variance approximately 1.